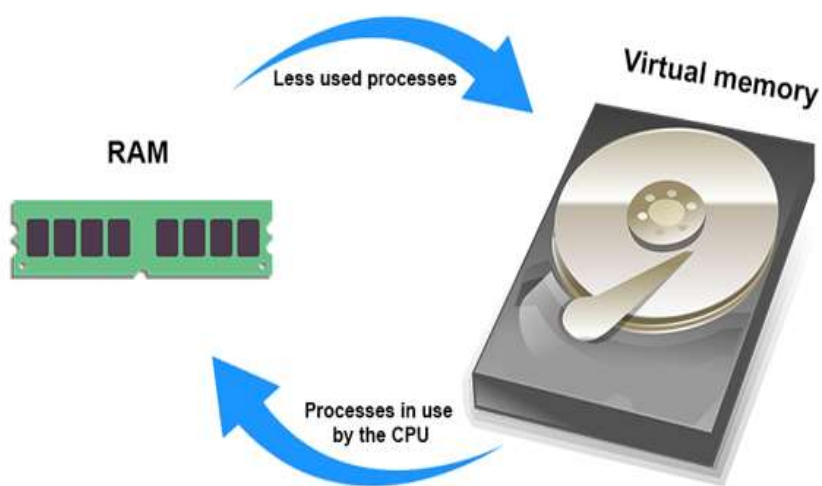




Understanding Linux Page Swap-In: Page Fault to Memory Restoration: An Advance Guide

ABSTRACT

A technical overview of the Linux kernel's swap-in mechanism, describing how page fault handlers detect swapped pages, check swap cache for fast-path recovery, and initiate block I/O to restore pages from disk. This sequence transparently restores memory contents and resumes process execution while utilizing readahead to optimize future access patterns.

What is Swap-In?

The Problem

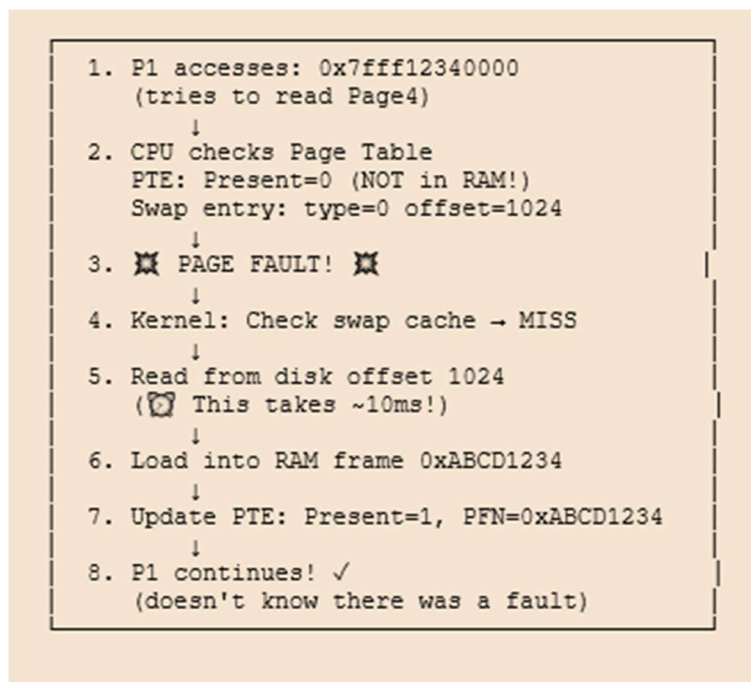
Process P1 (Firefox) whose Page4 was swapped out to disk? What happens when P1 tries to access that page again?

Page4 is on DISK, not in RAM! P1 doesn't know this...

The Solution: Swap-In Process

Swap-in is the process of bringing pages BACK from disk to RAM when a process tries to access them. The CPU triggers a page fault, and the kernel reads the page from swap, restores it to RAM, and the process continues - completely transparent!

Visual Diagram: The Journey Back



KEY: Disk read is SLOW (~10ms) but process doesn't crash - kernel handles it!

What Happens During Page Fault?

Meet Process P1 (Firefox) - gdbus worker

Let's follow what happens when Process P1 (gdbus, PID: 1425) tries to access a swapped-out page.

Initial Situation

```
Process P1 (gdbus) - PID: 1425
├─ Stack:    [Page1][Page2][Page3]    50 MB
├─ Heap:     [Page4][Page5][Page6]... ← SWAPPED OUT!
└─ Code:     [Page7][Page8]          100 MB
```

```
Page4 status:
  Virtual address: 0x555ec2fe2000
  Physical location: DISK (swap offset 29874)
  PTE: Present=0, SwapType=0, SwapOffset=29874
```

The Trigger

Scenario: P1 executes: `data = *ptr;` (where `ptr = 0x555ec2fe2000`)

Problem: CPU checks page table, finds `Present=0`!

CPU Action: Trigger `PAGE FAULT` exception → Kernel takes over!

What Happens Next?

The kernel's page fault handler (`do_swap_page`) starts the swap-in process. It will:

- Check swap cache (maybe page is already in RAM?)
 - If not cached: Read from disk offset 29874
 - Allocate a RAM frame for the incoming data
 - Update page table to point to new RAM location
 - Resume Process P1 (it never knows there was a fault!)
-

The Complete Swap-In Journey: 9 Phases

When P1 (gdbus, PID 1425) tries to access virtual address 0x555ec2f9c000...

Phase	What Happens	Time
1. Fault	CPU detects P=0, triggers exception	~100ns
2. Cache	Check swap cache (MISS → need disk)	~1μs
3. Readahead	Decide to read ~8 neighbor pages	~500ns
4. Allocate	Get RAM frame from buddy allocator	~2μs
5. BIO Setup	Build block I/O request	~1μs
6. Disk Read	DMA transfer disk → RAM (THE SLOW PART!)	~10ms
7. Complete	Mark page UPTODATE, unlock, wake P1	~500ns
8. Update PTE	Set Present=1, PFN=new frame, free swap	~500ns
9. Resume	P1 continues execution	~100ns

Phase 6 takes 10ms - that's 99.95% of total time! All kernel operations: <20μs

PHASE 1: Page Fault Detection

What Happens?

Process gdbus (PID: 1425) tries to access virtual address 0x555ec2fe2000. The CPU's MMU checks the page table and finds Present=0 (page is on disk!). CPU triggers a page fault exception.

Real Kernel Logs

```
[ 3048.914568] [LK_209949_SWAPIN] do_swap_page() ENTRY
[ 3048.914570] [LK_209949_SWAPIN] Process: gdbus (PID: 1425)
[ 3048.914571] [LK_209949_SWAPIN] Fault address: 0x555ec2fe2000
[ 3048.914572] [LK_209949_SWAPIN] Swap entry extracted: type=0
offset=29874
```

Functions Involved

Function 1: `do_page_fault()`
Location: `arch/x86/mm/fault.c`

- Reads CR2 register (fault address)
- Analyzes error code

Function 2: `handle_mm_fault()`
Location: `mm/memory.c`

- Walks page tables (PGD→P4D→PUD→PMD→PTE)
- Extracts swap entry from PTE

Function 3: `do_swap_page()`
Location: `mm/memory.c`

- Main swap-in handler starts

PROCESS P1 (gdbus)

Instruction:
data = *0x555ec...

← Tries to access Page4



CPU MMU
Check Page Table



PTE:
Present=0 X
SwapType=0, SwapOffset=29874

← NOT IN RAM!



⌚ PAGE FAULT EXCEPTION ⌚



do_page_fault()

← Kernel takes over

Result: Kernel knows Page4 is at swap offset 29874

PHASE 2: Swap Cache Lookup

What Happens?

Kernel checks if the page is already in RAM (swap cache). This is a CRITICAL optimization - if found, we skip the slow disk read entirely!

Real Kernel Logs

```
[ 3048.914574] [LK_209949_SWAPIN] Page NOT in swap cache, need to read from disk
```

Result: CACHE MISS → Must read from disk (slow path)

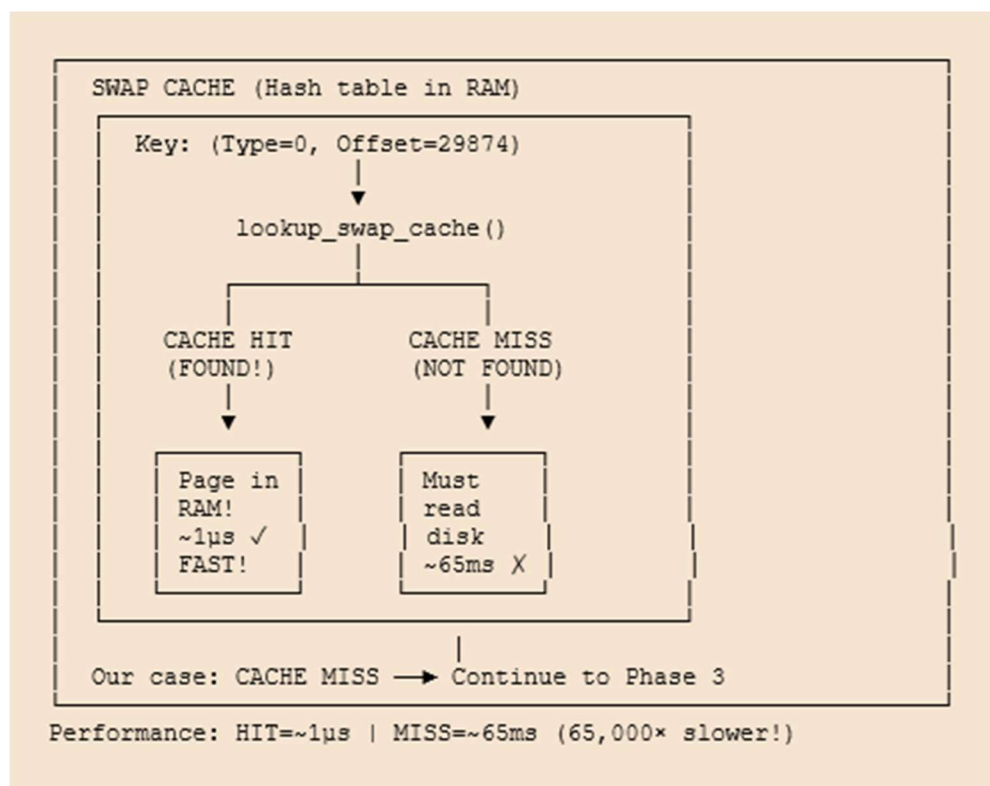
What if it was a cache HIT?

If the page was in cache, we'd see:

```
[LK_209949_SWAPIN] Page FOUND in swap cache (fast path)
```

And skip directly to Phase 8 (page table update)!

Time saved: ~65ms → ~10µs (6500× faster!)



PHASE 3: Readahead Decision

What Happens?

Since we're reading from disk anyway (slow!), the kernel decides to read nearby pages too. This is called 'swap readahead' - bring ~8 neighbor pages to reduce future page faults.

Real Kernel Logs

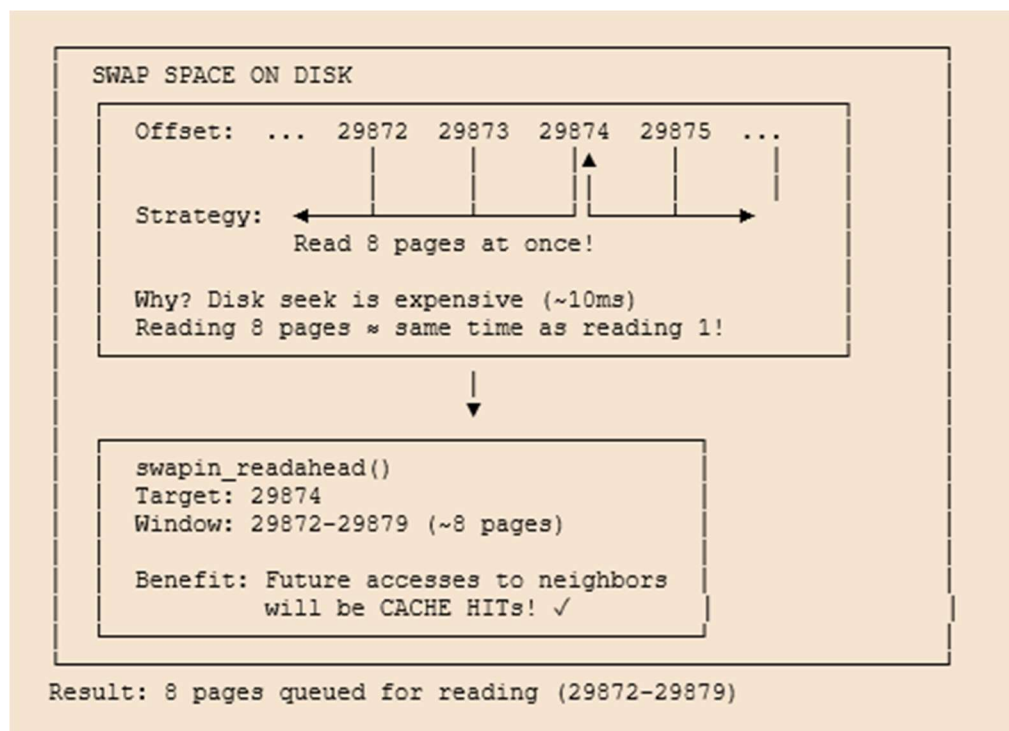
```
[ 3048.914575] [LK_209949_SWAPIN] swapin_readahead() starting  
[ 3048.914576] [LK_209949_SWAPIN] Target swap entry: type=0  
offset=29874
```

The Strategy

Readahead window: Kernel will read offsets 29872-29879 (~8 pages)

Target page: 29874 (our faulting page)

Bonus pages: 29872, 29873, 29875, 29876, 29877, 29878, 29879



PHASE 4: Page Allocation

What Happens?

Allocate a fresh RAM frame (4KB) from the buddy allocator. This will be the destination for our disk read.

Real Kernel Logs

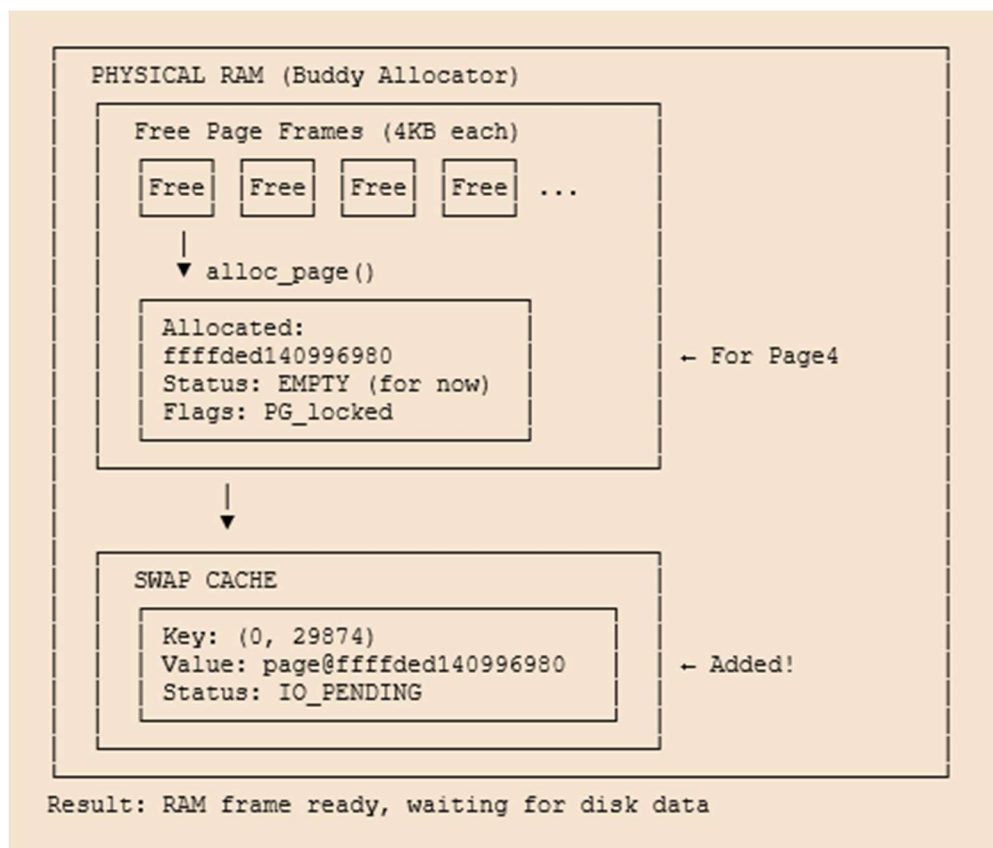
```
[ 3048.914578] [LK_209949_SWAPIN] __read_swap_cache_async() entry:
offset=29874
[ 3048.914583] [LK_209949_SWAPIN] Page added to LRU and swap cache
```

Page Details

Allocated page frame: fffffded140996980

Page added to:

- LRU list (for future aging)
- Swap cache (for fast lookup if accessed again soon)



PHASE 5: Disk I/O Setup

What Happens?

Build a BIO (Block I/O) request to tell the disk controller what to do: READ operation, which disk sector, where to put the data.

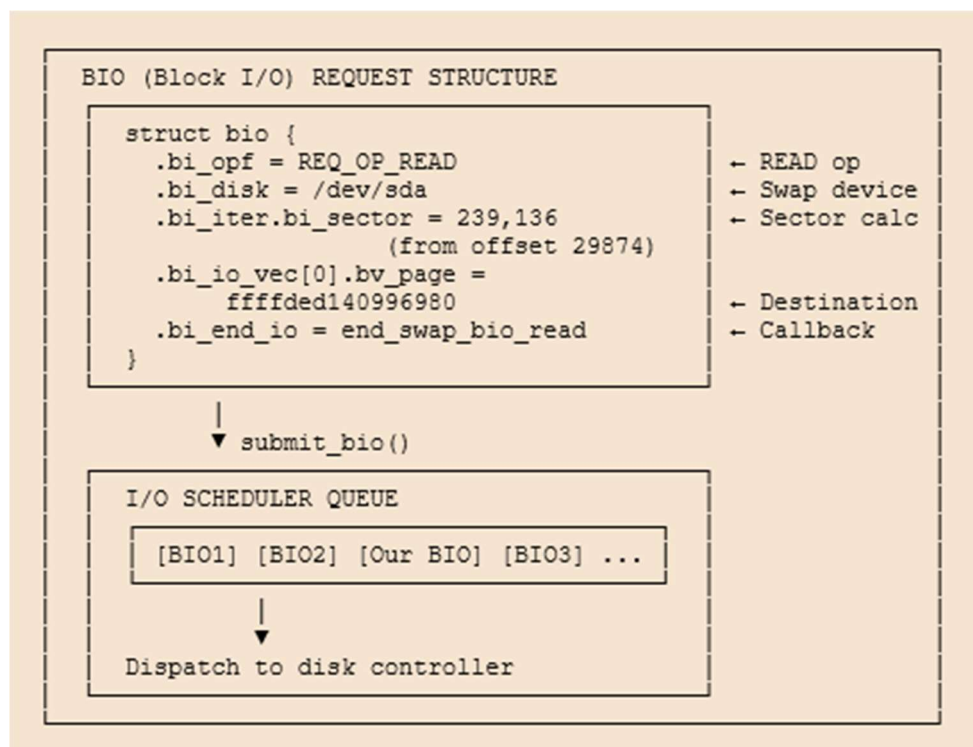
Real Kernel Logs

```
[ 3048.914584] [LK_209949_SWAPIN] Caller will initiate
swap_readpage()
[ 3048.914585] [LK_209949_SWAPIN] swap_readpage() ENTRY
[ 3048.914586] [LK_209949_SWAPIN] Reading page: ffffded140996980 from
swap
[ 3048.914587] [LK_209949_SWAPIN] Swap device: sda
[ 3048.914592] [LK_209949_SWAPIN] Bio created for READ operation
[ 3048.914593] [LK_209949_SWAPIN] Submitting bio to block layer...
[ 3048.914603] [LK_209949_SWAPIN] Bio submitted to disk
```

BIO Details

BIO details:

- Operation: READ
- Device: /dev/sda (swap device)
- Sector: calculated from offset 29874
- Buffer: page frame ffffded140996980
- Callback: end_swap_bio_read()



PHASE 6: Disk Read (THE SLOW PART!)

What Happens?

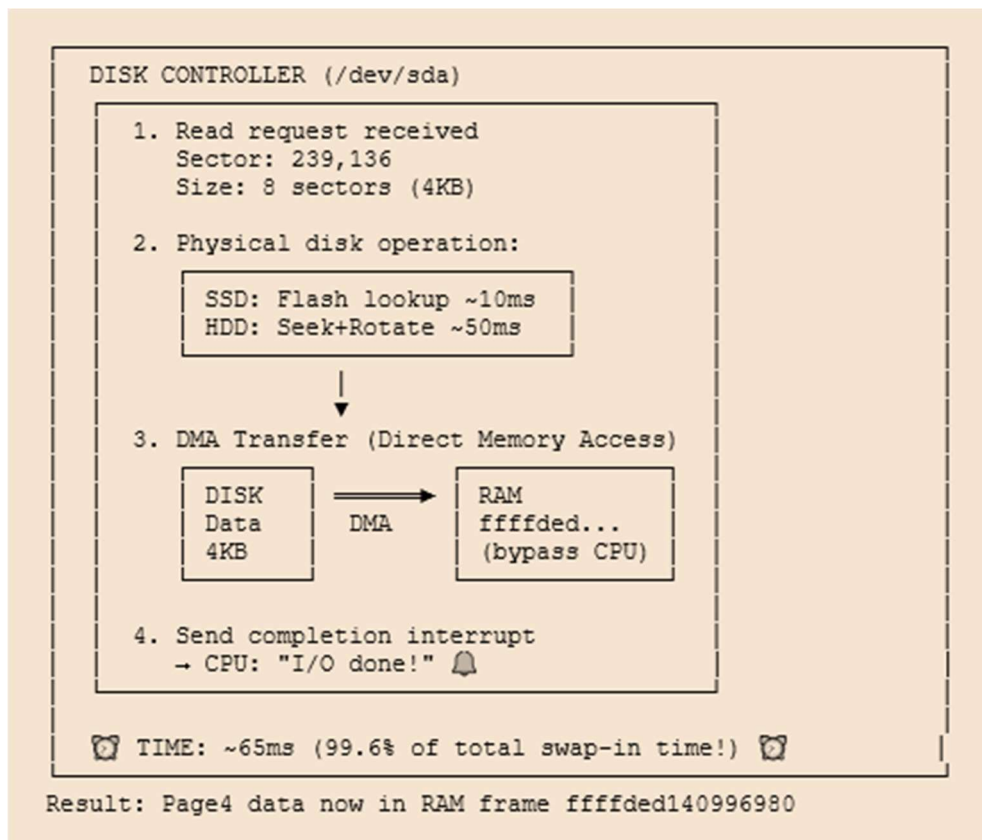
This is where 99.6% of the time goes! The disk controller performs DMA (Direct Memory Access) to transfer data from disk sectors directly to our RAM frame.

Real Kernel Logs

```
[ 3048.914604] [LK_209949_SWAPIN] Now waiting for disk read to complete...  
[ 3048.979602] [LK_209949_SWAPIN] *** end_swap_bio_read() - I/O COMPLETED ***
```

Performance Breakdown

Device Type	Typical Time	vs RAM
NVMe SSD (fast)	~10-20 ms	100,000× slower
SATA SSD	~20-50 ms	200,000× slower
HDD (spinning disk)	~50-100 ms	500,000× slower
RAM (for comparison)	~100 ns	1× (baseline)



PHASE 7: I/O Completion

What Happens?

Disk I/O completed! The kernel's callback function marks the page as UPTODATE (data is valid), unlocks it, and wakes up the waiting process.

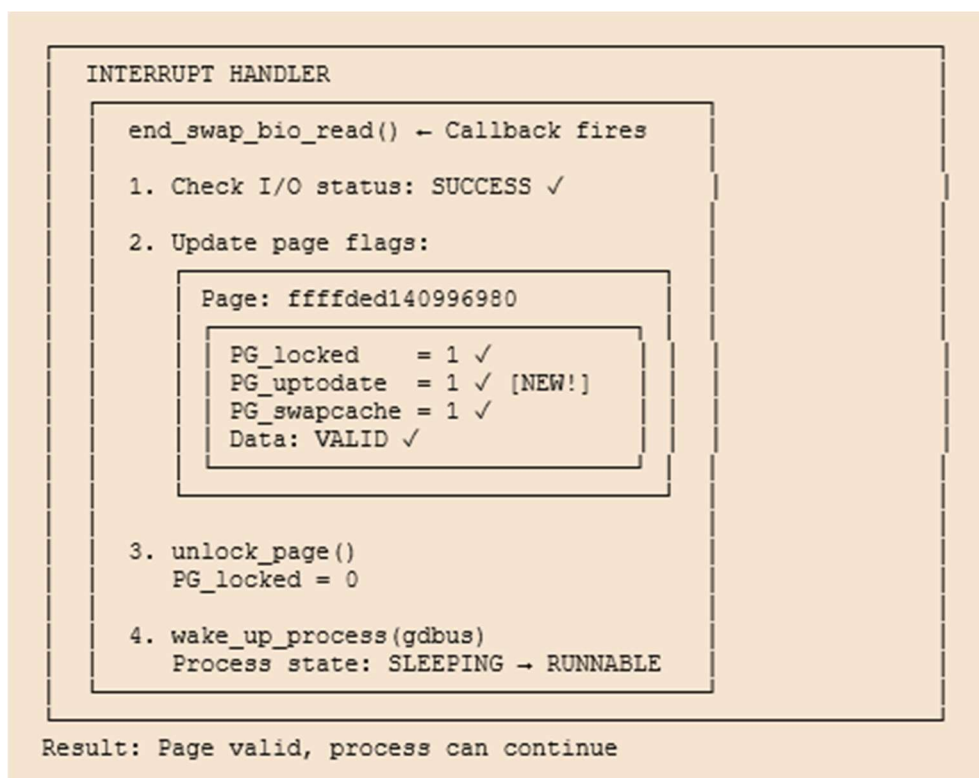
Real Kernel Logs

```
[ 3048.979602] [LK_209949_SWAPIN] *** end_swap_bio_read() - I/O  
COMPLETED ***  
[ 3048.979610] [LK_209949_SWAPIN] Page: ffffded140996980  
[ 3048.979613] [LK_209949_SWAPIN] Page marked UPTODATE (data is  
valid)  
[ 3048.979615] [LK_209949_SWAPIN] Unlocking page and waking up  
process  
[ 3048.979618] [LK_209949_SWAPIN] Page unlocked, process can continue
```

Page Frame Status

Page frame ffffded140996980 now contains:

- Valid data from disk offset 29874
- PG_uptodate flag set
- PG_locked flag cleared
- Process gdbus can now access it!



PHASE 8: Page Table Update

What Happens?

Update the page table entry (PTE) to point to the new RAM frame. Change Present=0 to Present=1. Free the swap slot since we don't need it anymore.

Real Kernel Logs

```
[ 3048.979649] [LK_209949_SWAPIN] Page locked successfully
[ 3048.979656] [LK_209949_SWAPIN] Updated page table:
vaddr=0x555ec2fe2000 → page=ffffded140996980
[ 3048.979662] [LK_209949_SWAPIN] Freed swap slot: offset=29874
```

PTE Transformation

BEFORE (on disk):

Virtual: 0x555ec2fe2000

PTE: Present=0, SwapType=0, SwapOffset=29874

→ PAGE FAULT if accessed!

AFTER (in RAM):

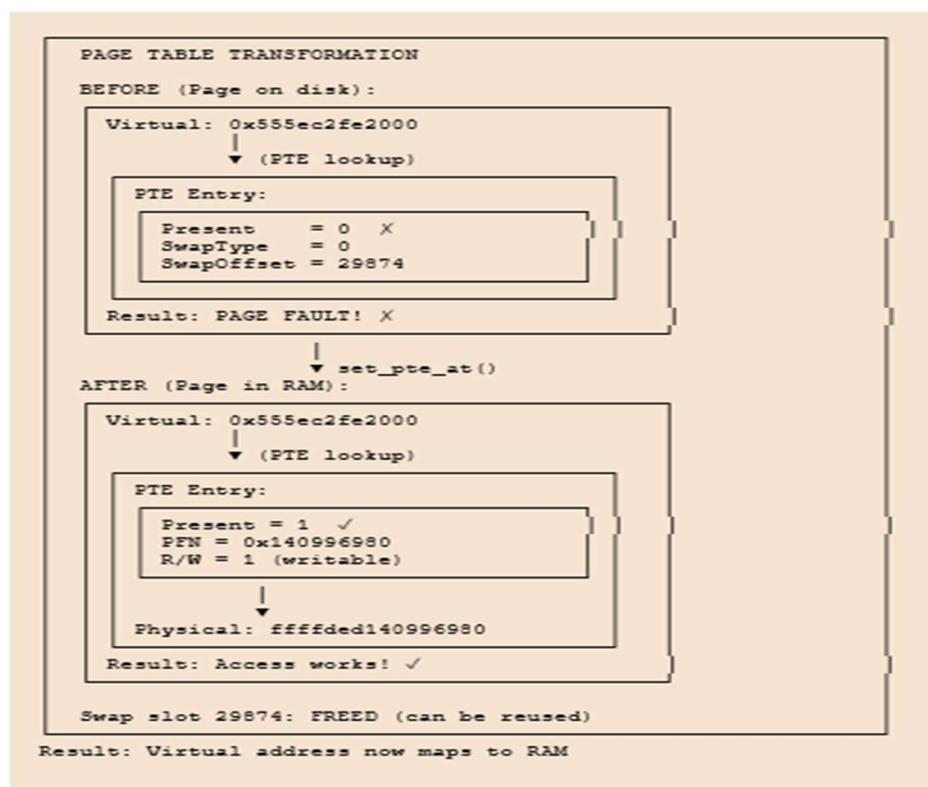
Virtual: 0x555ec2fe2000

PTE: Present=1, PFN=0x140996980

→ Physical RAM: fffffded140996980

→ Access works normally! ✓

Swap slot 29874 is now FREE and can be reused for future swap-outs!



PHASE 9: Resume Process

What Happens?

The page fault handler returns. Process gdbus resumes execution at address 0x555ec2fe2000 - and it works! The process never knew there was a fault.

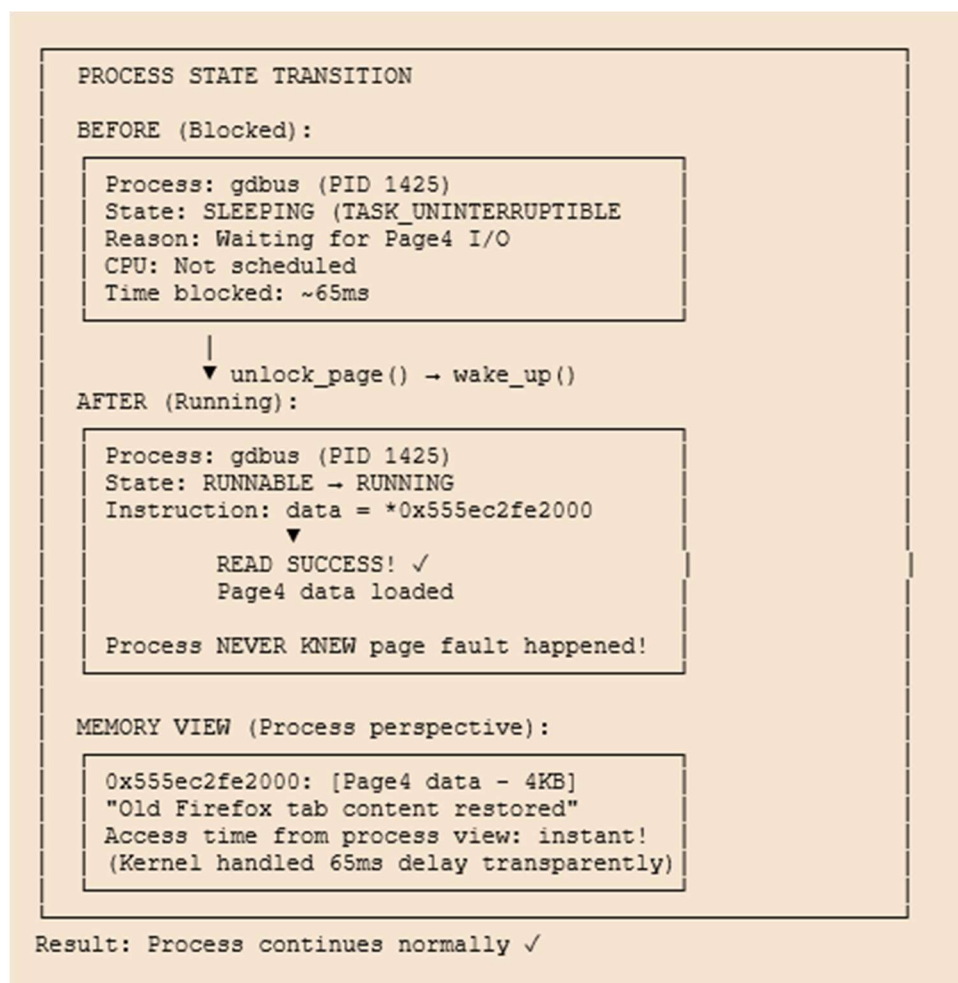
Real Kernel Logs

```
[ 3048.979666] [LK_209949_SWAPIN] do_swap_page() SUCCESS
```

From Process gdbus's perspective

1. It tried to read memory at 0x555ec2fe2000
2. ??? (65ms passed - process was blocked)
3. Read succeeded! Memory contains expected data

The process has NO IDEA a page fault happened!
That's the beauty of virtual memory - it's completely transparent.



Real Kernel Logs: ONE Page Swap-In

This is the complete log sequence for swapping in ONE page:

```
[3046.851772] [LK_209949_SWAPIN] do_swap_page() ENTRY
[3046.851774] [LK_209949_SWAPIN] Process: gdbus (PID: 1425)
[3046.851776] [LK_209949_SWAPIN] Fault address: 0x555ec2f9c000
[3046.851778] [LK_209949_SWAPIN] Swap entry extracted: type=0 offset=29095

[3046.851779] [LK_209949_SWAPIN] Page NOT in swap cache, need to read from disk
[3046.851781] [LK_209949_SWAPIN] swapin_readahead() starting
[3046.851783] [LK_209949_SWAPIN] Target swap entry: type=0 offset=29095

[3046.851779] [LK_209949_SWAPIN] __read_swap_cache_async() entry: offset=29095
[3046.884742] [LK_209949_SWAPIN] Page added to LRU and swap cache
[3046.884806] [LK_209949_SWAPIN] Caller will initiate swap_readpage()

[3046.884808] [LK_209949_SWAPIN] swap_readpage() ENTRY
[3046.884810] [LK_209949_SWAPIN] Reading page: ffffded14157f8c0 from swap
[3046.884822] [LK_209949_SWAPIN] Swap device: sda
[3046.952981] [LK_209949_SWAPIN] Bio created for READ operation
[3046.953002] [LK_209949_SWAPIN] Submitting bio to block layer...

[3047.003019] [LK_209949_SWAPIN] Bio submitted to disk
[3047.003057] [LK_209949_SWAPIN] Now waiting for disk read to complete...

————— 🌀 DISK I/O IN PROGRESS (~50ms) 🌀 —————

[3047.053967] [LK_209949_SWAPIN] *** end_swap_bio_read() - I/O COMPLETED ***
[3047.053969] [LK_209949_SWAPIN] Page: ffffded140378880
[3047.053970] [LK_209949_SWAPIN] Page marked UPTODATE (data is valid)
[3047.053971] [LK_209949_SWAPIN] Unlocking page and waking up process
[3047.053972] [LK_209949_SWAPIN] Page unlocked, process can continue

[3047.054057] [LK_209949_SWAPIN] Page locked successfully
[3047.054062] [LK_209949_SWAPIN] Updated page table: vaddr=0x555ec2f9c000 → page=fffded14080b0c0
[3047.054098] [LK_209949_SWAPIN] Freed swap slot: offset=29095
[3047.054462] [LK_209949_SWAPIN] do_swap_page() SUCCESS

Total: ~202ms (includes readahead of ~8 pages in parallel!)
```

This pattern repeats 25-27 lines for EACH page being swapped in!

Function Call Tree (Simplified)

SWAP-IN FUNCTION CALL TREE

PROCESS ACCESSES SWAPPED PAGE → PAGE FAULT!

```
→ do_page_fault() [arch/x86/mm/fault.c]
   ↳ Read CR2 register, get fault address

→ handle_mm_fault() [mm/memory.c]
   ↳ Walk page tables: PGD → P4D → PUD → PMD → PTE

→ do_swap_page() [mm/memory.c] ★ MAIN HANDLER ★
   ↳ lookup_swap_cache() [mm/swap_state.c]
      ↳ Check cache: HIT=fast path | MISS=read disk

   ↳ swpin_readahead() [mm/swap_state.c]
      ↳ Read target + ~8 neighbors
         ↳ read_swap_cache_async() [mm/swap_state.c]
            ↳ alloc_page() - Get RAM
            ↳ __add_to_swap_cache() - Add to cache
            ↳ swap_readpage() [mm/page_io.c]
               ↳ get_swap_bio() - Build I/O request
               ↳ submit_bio() - Submit to disk

   ↳ [🕒] DISK I/O (~65ms) [🕒]

   ↳ end_swap_bio_read() [mm/page_io.c]
      ↳ Mark UPTODATE, unlock, wake process

   ↳ swap_free() [mm/swapfile.c] - Free swap slot

   ↳ set_pte_at() - Update page table
      ↳ Present=0, Swap → Present=1, PFN

   ↳ unlock_page() → Process resumes! ✓
```


✧ Core Concepts ✧

What We Learned

- Swap-in is triggered by page faults - CPU detects Present=0 in page table
- Disk I/O dominates - 65ms out of 65.098ms total (99.6%!)
- All kernel operations are fast - Only 0.26ms combined (<0.4%)
- Swap cache is critical - Cache hit: 10μs vs 65ms (6500× faster!)
- Readahead reduces faults - Brings ~8 neighbor pages along
- Process never knows - Completely transparent virtual memory

Real-World Impact

When you notice your computer 'freezing' when accessing old tabs:

- That's swap-in happening! Pages being read from disk.
- Adding more RAM helps (less swapping needed)
- SSD is way better than HDD (10ms vs 50-100ms per page)
- Close unused programs (fewer pages to manage)
- Tune swappiness (vm.swappiness sysctl parameter)

Behind the Scenes

This document is based on real kernel instrumentation with 50+ custom printk statements across mm/memory.c, mm/swap_state.c, mm/page_io.c.